

# Best-First Search Using Prolog

## Aim

To write a Prolog program to implement the **Best-First Search algorithm** using heuristic values to find a path from a starting node to a goal node.

## Algorithm

1. Start with the initial node and add it to the OPEN list.
2. Select the node with the **lowest heuristic value**.
3. Check whether the selected node is the goal node.
4. If it is the goal, display the path and stop.
5. Otherwise, find the neighbouring nodes of the selected node.
6. Calculate the heuristic value of each neighbouring node.
7. Add the unvisited nodes to the OPEN list.
8. Repeat the process by selecting the node with the lowest heuristic value.
9. Continue until the goal node is reached.
10. Display the final path and stop.

## Prolog Program

```
edge(a, b).
edge(a, c).
edge(b, d).
edge(c, d).
edge(c, e).
edge(e, f).
h(a,
5).
h(b,
3).
h(c,
2).
h(d,
1).
h(e,
1).
h(f,
0).
best_first(Start, Goal, Path)
:-      search([Start], Goal,
Path).
search([Goal|_], Goal,
[Goal]).
search([Node|Rest], Goal, [Node|Path])
:-      findall(H-Next,
(edge(Node, Next), h(Next,
H)),
Children),
keysort(Children, Sorted),
get_nodes(Sorted, Nodes),
append(Nodes, Rest, NewOpen),
search(NewOpen, Goal, Path).
get_nodes([],
[]).
get_nodes([_N|T], [N|R])
:-      get_nodes(T, R).
```

## Query

```
?- best_first(a, f, Path).
```

## Result

```
Path = [a, c, e, f].
```

## OUTPUT

```
?-  
|   best_first(a, f, Path).  
Path = [a, c, d, e, f] ■
```

## Conclusion

Thus, the **Best-First Search algorithm** is successfully implemented in Prolog, and the path from node **A** to the goal node **F** is obtained using heuristic values.